A Project Report on

GESTURE-BASED SMART TRAFFIC SIGNAL CONTROL SYSTEM USING 8051 MICROCONTROLLER

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

JANJANAM BABU VENKATA GANESH
(24BQ1A0451)

JANGAM HARI HARAN
(24BQ1A0449)

KOLLI RANJITH KUMAR
(24BQ1A0466)

AVVARI SRI ADITHYA BHARADWAZ
(24BQ1A0410)

Under the Esteemed Guidance of

Mr. B. Suresh Babu, M. Tech.,



VASIREDDY VENKATADRI
INSTITUTE OF TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
NAMBURU

(Approved by AICTE, New Delhi)
(Permanently Affiliated to Jawaharlal Nehru Technological University, Kakinada (JNTUK))
(Accredited by NAAC and NBA (Eligible Programmes))

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
NAMBURU

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

(Approved by AICTE, New Delhi)
(Permanently Affiliated to Jawaharlal Nehru Technological University, Kakinada (JNTUK))
(Accredited by NAAC and NBA (Eligible Programmes))



CERTIFICATE

This is to certify that the project report entitled "GESTURE-BASED SMART TRAFFIC SIGNAL CONTROL SYSTEM USING 8051 MICROCONTROLLER" is a bonafide work carried out by JANJANAM BABU VENKATA GANESH (24BQ1A0451), AVVARI SRI ADITHYA BHARADWAZ (24BQ1A0410),JANGAM HARI HARAN (24BQ1A0449),KOLLI RANJITH KUMAR (24BQ1A0466)) under my guidance and supervision, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering of [University Name] during the academic year [Academic Year].

Viva-Voce Date: _____

Internal Guide **Head of the Department**

(B.Mahidhar)

External Examiner

ACKNOLEDGEMENT

The successful completion of this project would not have been possible without the guidance, support, and encouragement of several individuals, to whom we express our sincere gratitude.

We are deeply thankful to our project guide, [**B. Mahidhar**], for the invaluable guidance, constant encouragement, and technical insight provided throughout every stage of this project, from conceptualisation of the gesture-based traffic control idea to the final testing of the prototype.

We are grateful to the principal and the management of [VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY,NAMBUR] for their continuous support and for providing excellent infrastructure for carrying out this project.

We also thank the teaching and non-teaching staff of the Department of Electronics and Communication Engineering for their cooperation and assistance throughout the course of this project.

Finally, we thank our families and friends for their unwavering encouragement and support during the completion of this work.

Project Associate(s)

JANJANAM VENKATA BABU GANESH (24BQ1A0451)
AVVARI SRI ADITHYA BHARADWAZ (24BQ1A0410)
JANGAM HARI HARAN (24BQ1A0449)
KOLLI RANJITH KUMAR (24BQ1A0466)

Declaration

This is to certify that the thesis titled **"GESTURE-BASED SMART TRAFFIC SIGNAL CONTROL SYSTEM USING 8051 MICROCONTROLLER"** is a bonafide work done by me, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** and submitted to the Department of Electronics & Communication Engineering, Vasireddy Venkatadri Institute of Technology, Namburu**.**

I also declare that this project is a result of my own effort and that has not been copied from anyone and I have taken only citations from the sources which are mentioned in the references.

This work was not submitted earlier at any other University or institute for the award of any degree.

AVVARI SRI ADITHYA BHARADWAZ
Regd no:24BQ1Q0410

Place: Nambur

CONTENTS

TOPIC NAME	Page No.
LIST OF FIGURES	**I**
LIST OF TABLES	**II**
ABSTRACT	**IIII**
CHAPTER 1 INTRODUCTION TO EMBEDDED	**11**
1.1 Overview of Embedded Systems	**12**
1.2 Characteristics and features of Embedded Systems	**12**
1.3 Classification of Embedded Systems	**12**
1.3.1 Stand-alone Embedded Systems	13
1.3.2 Real-Time Embedded Systems	13
1.3.3 Networked Embedded Systems	13
1.3.4 Mobile Embedded Systems	**13**
1.4 Applications of Embedded Systems	**13**
1.5 Importance of E.S in Smart Traffic Management	**14**
1.6 Role of E.S in this Project	**14**
CHAPTER 2 INTRODUCTION TO MICROCONTROLLER	**15**
2.1 What is a Microcontroller?	**15**
2.2 The 8051 Microcontroller Family	**15**
2.3 Features of the AT89S51/AT89C51 Microcontroller	**15**
2.4 Pin Diagram and Pin Description	**16**
2.5 Internal Architecture of the 8051	**16**
2.6 Memory Organization	**17**
2.7 Timers and counters	**17**

2.8 Interrupts 17

2.9 Register Set 17

2.10 Advantages of Using the 8051 Microcontroller 18

CHAPTER 3 LITERATURE SURVEY 19

3.1 Introduction 19

3.2 Conventional Fixed-Timer Traffic Signal Systems 19

3.3 Embedded Microcontroller-Based Traffic Control 19

3.4 Gesture Recognition for Human-Machine Interaction 19

3.5 Image Processing and AI-Based Traffic Control 20

3.6 IoT-Enabled Smart Traffic Systems 20

3.7 Comparative Summary and Research Gap 20

CHAPTER 4 INTRODUCTION TO PROJECT 22

4.1 Project Statement 22

4.2 Need for the Project 22

4.3 Objectives of the Project 22

4.4 Existing Systems 22

4.5 Limitations of the Existing System 23

4.6 proposed System 23

4.7 Advantages of the Proposed System 23

4.8 Applications 23

4.9 System Overview 24

CHAPTER 5 PROJECT IMPLEMENTATION 25

5.1 Hardware Description 25

5.2 Software Description 25

5.3 System Block Diagram **25**

5.4 Working Principle **26**

5.5 Algorithm **26**

5.6 Flowchart **27**

5.7 Embedded C Program Explanation **27**

5.8 Circuit Connections and Port Allocation **28**

5.9 Traffic Signal Logic Summary **29**

CHAPTER 6 RESULTS **30**

6.1 Proteus Circuit Diagram **30**

6.2 North Road green Output **30**

6.3 South Road green Output **31**

6.4 East Road green Output **31**

6.5 West Road green Output **32**

6.6 Emergency Mode-All Roads Red **32**

6.7 Summary of Results **33**

CHAPTER 7 CONCLUSION AND FUTURE SCOPE **34**

7.1 Conclusion **34**

7.2 Advantages **34**

7.3 Limitations **34**

7.4 Future Scope **34**

CHAPTER 8 REFERENCES **35**

LIST OF FIGURES

Figure No.	Description
Fig 2.1	Pin Diagram of AT89C51 Microcontroller
Fig 2.2	Internal Architecture (Block Diagram) of 8051
Fig 5.1	System Block Diagram
Fig 5.2	Algorithm Flowchart
Fig 6.1	Proteus Circuit Diagram of the Complete System
Fig 6.2	North Road Green Output
Fig 6.3	South Road Green Output
Fig 6.4	East Road Green Output
Fig 6.5	West Road Green Output
Fig 6.6	Emergency Mode – All Roads Red

LIST OF TABLES

Table No.	Description
Table 5.1	Push Button to Road/Function Mapping
Table 5.2	Port Pin Allocation
Table 6.1	Summary of Observed Output States

ABSTRACT

Traffic congestion at road junctions is one of the most persistent challenges faced by urban infrastructure today. Conventional traffic signal controllers operate on fixed timers that do not adapt to real-time traffic conditions, resulting in unnecessary waiting times, fuel wastage, and difficulty in handling emergency situations. This project, titled "Gesture-Based Smart Traffic Signal Control System using 8051 Microcontroller", proposes an embedded system that allows a traffic controller — such as a traffic police officer — to manually direct traffic flow at a four-road junction through simple hand gesture commands, while also providing a dedicated emergency mode to instantly halt traffic on all roads.

The system is built around the AT89S51/AT89C51 8051 microcontroller, which is interfaced with Red, Yellow, and Green LED indicators for each of the four roads (North, South, East, and West) at the junction. In the current working prototype, five push buttons are used to emulate hand gesture inputs for demonstration and validation purposes, since implementing a complete camera-based hand gesture recognition pipeline requires additional hardware such as an ESP32-CAM module, a Raspberry Pi, and computer vision frameworks such as OpenCV, MediaPipe, or TensorFlow Lite. It is important to note that this project is not merely a push-button traffic light controller — it is a functional embedded prototype that demonstrates and validates the complete gesture-controlled traffic signal logic, control algorithm, and hardware interfacing that would underpin a full gesture-recognition-based deployment.

Each of the first four buttons, when pressed, activates the Green signal on one specific road while forcing the Red signal on the remaining three roads, thereby giving right-of-way to a single direction at a time under manual control. The fifth button activates an Emergency Mode, in which all four roads are simultaneously switched to Red, halting traffic in every direction to allow emergency vehicles or pedestrians safe passage. The complete system was designed and simulated using Proteus Professional, with the embedded control firmware developed in Embedded C using Keil µVision.

The results obtained from the Proteus simulation confirm that the system correctly implements the intended gesture-to-signal mapping for all five operating modes. The project also outlines a future scope in which the push-button interface can be replaced by a genuine hand gesture recognition module using a camera, ESP32-CAM, or Raspberry Pi running OpenCV/MediaPipe-based gesture classification models, enabling contactless, intelligent, and human-directed control of traffic signals — a step toward safer and smarter city traffic infrastructure.

CHAPTER 1

INTRODUCTION TO EMBEDDED SYSTEMS

1.1 Overview of Embedded Systems

An embedded system is a combination of computer hardware and software, and often additional mechanical or other parts, designed to perform a dedicated function within a larger mechanical or electrical system. Unlike a general-purpose computer, such as a personal computer, which is designed to perform a wide range of tasks, an embedded system is designed to perform a specific, often narrowly defined task, and it is usually embedded as part of a complete device that includes hardware and mechanical parts. Embedded systems form the backbone of nearly every modern electronic product, from household appliances to industrial automation equipment, medical instruments, automotive control units, and, as demonstrated in this project, intelligent traffic management infrastructure.

An embedded system typically consists of a microcontroller or microprocessor core, memory (program and data), input/output interfaces, and often sensors or actuators, all working together under the control of firmware written specifically for the target application. Because embedded systems are designed for specific tasks, engineers can optimize them to reduce the size and cost of the product, improve reliability and performance, and in many cases dramatically reduce power consumption compared to a general-purpose computing solution.

1.2 Characteristics and Features of Embedded Systems

- Single-functioned: An embedded system usually performs a specialized operation and does it repeatedly, unlike a desktop computer that performs many different operations.
- Tightly constrained: Embedded systems typically have tight constraints on design metrics such as cost, size, power consumption, and performance.
- Reactive and real-time: Many embedded systems must continually react to changes in the system's environment and must compute results in real time without delay.
- Reliability: Embedded systems are often deployed in critical applications and are expected to run continuously for long durations without failure.
- Microcontroller-based: Most embedded systems are built around a microcontroller which integrates the CPU, memory, and I/O peripherals on a single chip.
- Direct hardware interfacing: Embedded systems interact directly with sensors, actuators, LEDs, motors, and other physical hardware components.

1.3 Classification of Embedded Systems

Embedded systems can be broadly classified based on performance and functional requirements into the following categories:

1.3.1 Stand-alone Embedded Systems

These systems take input in the form of electrical signals from transducers or commands from a human operator, process it, and produce the desired output. The entire operation is self-contained on a single device. Examples include digital cameras, microwave ovens, and, in a simplified sense, the traffic signal controller developed in this project, which independently reads push-button (gesture) inputs and drives LED outputs without depending on an external computer.

1.3.2 Real-Time Embedded Systems

A real-time embedded system is designed to perform a specific task within a defined time constraint. Real-time systems are further divided into hard real-time systems, where missing a deadline can cause catastrophic failure (e.g., airbag control systems), and soft real-time systems, where an occasional missed deadline is tolerable (e.g., traffic light controllers, where minor timing variance does not lead to catastrophic outcomes).

1.3.3 Networked Embedded Systems

These systems are connected to a network to provide output to attached devices or to communicate status information. Modern smart traffic systems increasingly rely on networked embedded architectures — for example, IoT-enabled traffic controllers that report congestion data to a central traffic management server, a direction discussed further in the Future Scope of this project (Chapter 7).

1.3.4 Mobile Embedded Systems

These are small-sized embedded systems designed for portability, such as mobile phones, digital cameras, and MP3 players, where size and power consumption are the most restrictive design constraints.

1.4 Applications of Embedded Systems

- Consumer electronics: washing machines, microwave ovens, digital cameras, air conditioners.
- Automotive systems: engine control units, anti-lock braking systems, airbag deployment systems.
- Telecommunications: routers, modems, mobile handsets.
- Medical electronics: patient monitoring systems, infusion pumps, diagnostic equipment.
- Industrial automation: programmable logic controllers, robotic arms, process control systems.
- Traffic and transportation systems: traffic light controllers, vehicle counting systems, smart parking systems, and gesture/AI-based traffic management systems such as the one developed in this project.

1.5 Importance of Embedded Systems in Smart Traffic Management

Traffic congestion is a growing challenge in urban areas across the world, and conventional fixed-timer traffic signal systems are often unable to respond effectively to real-time traffic conditions or emergency situations. Embedded systems play a pivotal role in the evolution of smart traffic management because they enable low-cost, dedicated, real-time control of traffic signal hardware while remaining compact and power-efficient enough to be deployed at every road junction.

By using a microcontroller-based embedded platform, it becomes possible to implement flexible signal control logic — such as manually overriding automatic signal timing through operator gestures, or instantly triggering an all-red emergency state — without the cost and complexity of a full computer-based traffic management system. This makes embedded systems the ideal foundation for the gesture-based smart traffic signal controller presented in this report, and also the natural platform on which more advanced AI-based gesture recognition and computer-vision-driven traffic control can be built in the future.

1.6 Role of Embedded Systems in This Project

In this project, the AT89S51/AT89C51 8051 microcontroller functions as the embedded core of the traffic control system. It continuously monitors five push buttons that emulate gesture commands, and based on the specific button pressed, it drives twelve LEDs (Red, Yellow, and Green for each of the four roads) to reflect the appropriate signal state at the junction. The microcontroller executes this control logic in real time, with negligible latency between the gesture command and the corresponding change in the traffic signal display, illustrating the reactive, dedicated, and reliable nature that defines embedded systems.

CHAPTER 2

INTRODUCTION TO MICROCONTROLLER

2.1 What is a Microcontroller?

A microcontroller is a compact integrated circuit designed to govern a specific operation in an embedded system. A typical microcontroller includes a processor core, memory (RAM and ROM/Flash), and programmable input/output peripherals, all fabricated on a single chip. Because it combines all the essential components of a computer on one chip, a microcontroller is often referred to as a "computer on a chip." Microcontrollers are widely used in automatically controlled products and devices, such as automobile engine control systems, remote controls, office machines, appliances, and, as in this project, embedded traffic signal controllers.

2.2 The 8051 Microcontroller Family

The 8051 is an 8-bit microcontroller originally developed by Intel in 1980 for use in embedded systems. It quickly became an industry standard due to its simple architecture, ease of programming, and low cost, and it continues to be widely taught and used in academic and industrial embedded system projects even today. Numerous semiconductor manufacturers, including Atmel (now Microchip), have produced enhanced and pin-compatible variants of the original 8051 core, such as the AT89C51 and AT89S51 devices used in this project. These devices retain the original 8051 instruction set while adding features such as in-system programmable Flash memory, making them convenient for rapid prototyping and reprogramming during development.

2.3 Features of the AT89S51/AT89C51 Microcontroller

- 8-bit CPU optimized for control applications, compatible with the industry-standard MCS-51 instruction set.
- 4 KB of on-chip re-programmable Flash memory for program storage, supporting up to 1,000 write/erase cycles.
- 128 bytes of internal RAM for data storage.
- Four 8-bit bidirectional I/O ports (P0, P1, P2, P3), providing 32 I/O lines.
- Two 16-bit Timer/Counter modules (Timer 0 and Timer 1).
- A full-duplex serial UART communication port.
- Six interrupt sources with two priority levels.
- On-chip oscillator and clock circuitry, typically operated with an external crystal (commonly 11.0592 MHz).
- Low power idle and power-down modes for power-sensitive applications.
- In-System Programming (ISP) capability in the AT89S51 variant, allowing the chip to be reprogrammed without removing it from the circuit.

2.4 Pin Diagram and Pin Description

The AT89C51/AT89S51 is available in a 40-pin DIP (Dual In-line Package). The key pin groups are summarized below:

Pin(s)	Function
1–8 (P1.0–P1.7)	Port 1 — general-purpose bidirectional I/O port (used in this project to drive road-signal LED groups)
9	RST — active-high Reset input
10–17 (P3.0–P3.7)	Port 3 — bidirectional I/O with alternate functions (INT0, INT1, T0, T1, RxD, TxD, WR, RD)
18, 19	XTAL2, XTAL1 — crystal oscillator connections
20	GND — ground reference
21–28 (P2.0–P2.7)	Port 2 — bidirectional I/O port (used in this project to drive additional road-signal LED groups and, in larger systems, the high-order address bus)
29	PSEN — Program Store Enable (used for external program memory)
30	ALE/PROG — Address Latch Enable
31	EA/VPP — External Access enable
32–39 (P0.0–P0.7)	Port 0 — bidirectional I/O port / multiplexed address-data bus
40	VCC — +5V supply voltage

2.5 Internal Architecture of the 8051

The internal architecture of the 8051 microcontroller comprises the following major functional blocks:

- CPU (Central Processing Unit): performs arithmetic and logic operations on 8-bit data, controlled by the instruction decoder and timing/control unit.
- Program Counter (PC) and Data Pointer (DPTR): the PC holds the address of the next instruction to execute; the DPTR is used for indirect addressing of external memory.
- Internal RAM (128 bytes): used for storing temporary variables, the stack, and general-purpose registers.
- Special Function Registers (SFRs): a set of registers mapped to specific addresses that control and monitor peripherals such as timers, ports, and the serial interface.
- Four I/O Ports (P0–P3): provide the interface between the microcontroller and external hardware such as LEDs, switches, and other peripherals.
- Two 16-bit Timers/Counters (Timer 0 and Timer 1): used for generating precise time delays or counting external events — in this project, timers are used to implement the deterministic signal-hold delays.

- Serial Port (UART): enables serial communication, useful for future upgrades such as transmitting traffic status to a monitoring system.
- Interrupt Control Unit: manages five (or six, depending on variant) interrupt sources including external interrupts, timer interrupts, and the serial interrupt, with two selectable priority levels.
- Oscillator and Clock Circuit: generates the internal clock signal from an external crystal, typically 11.0592 MHz, which determines instruction execution speed.

2.6 Memory Organization

The 8051 architecture follows a Harvard architecture with separate address spaces for program memory and data memory. Program memory (Flash/ROM) stores the compiled Embedded C firmware that implements the gesture-to-signal control logic, while internal RAM stores runtime variables such as the current active road and button-debounce counters. The 8051 also supports external data memory (up to 64 KB) through Ports 0 and 2 acting as a multiplexed address/data bus, although this project uses only internal memory resources since the control logic and I/O requirements are modest.

2.7 Timers and Counters

The 8051 provides two 16-bit Timer/Counter units, Timer 0 and Timer 1, each of which can operate as either a timer (counting internal clock pulses) or a counter (counting external events on the T0/T1 pins). These timers can be configured in several modes, including 13-bit, 16-bit auto-reload, and 8-bit auto-reload modes. In embedded traffic controller designs, timers are commonly used to implement fixed-duration signal phases; in this gesture-controlled prototype, timer-based software delay routines are used to ensure clean, debounced button reads and stable signal transitions.

2.8 Interrupts

Interrupts allow the 8051 to respond immediately to specific internal or external events without continuously polling for them, improving responsiveness and efficiency. The 8051 supports two external interrupts (INT0, INT1), two timer interrupts (Timer 0 and Timer 1 overflow), and a serial port interrupt. While the current prototype primarily uses polled I/O for simplicity and predictability during Proteus simulation, the external interrupt pins provide a natural upgrade path for connecting a future gesture-recognition module (such as an ESP32-CAM) that could signal detected gestures to the 8051 via an interrupt line.

2.9 Register Set

The 8051 register set includes the Accumulator (A), the B register (used in multiply/divide operations), the Program Status Word (PSW) which holds condition flags such as Carry, Auxiliary Carry, and Overflow, the Stack Pointer (SP), the Data Pointer (DPTR), and four banks of eight general-purpose registers (R0–R7) that can be selectively activated through the

PSW register bank-select bits. This flexible register architecture makes context switching and interrupt handling efficient even with the 8051's modest 128 bytes of internal RAM.

2.10 Advantages of Using the 8051 Microcontroller

- Simple, well-documented architecture that is easy to program in Embedded C or Assembly.
- Extremely low cost, making it suitable for cost-sensitive, mass-deployable applications such as traffic signal controllers.
- Wide industry and academic support, with abundant development tools, simulators (such as Proteus), and compilers (such as Keil µVision).
- Sufficient I/O lines (32 total) to directly drive multiple LED groups without requiring additional port-expander hardware, as demonstrated in this project's four-road, twelve-LED signal array.
- Proven reliability in real-time control applications, with deterministic instruction timing.
- In-System Programmable variants (AT89S51) allow rapid firmware iteration during development and testing.

CHAPTER 3

LITERATURE SURVEY

3.1 Introduction

Traffic signal control has evolved considerably over the last few decades, moving from simple fixed-timer electromechanical controllers to microcontroller and microprocessor based systems, and more recently toward AI-driven and IoT-connected smart traffic infrastructure. This chapter reviews the major approaches reported in the literature and industry practice that are relevant to the gesture-based smart traffic signal controller developed in this project, and positions the proposed system with respect to these existing approaches.

3.2 Conventional Fixed-Timer Traffic Signal Systems

The earliest and still most widely deployed traffic signal systems use fixed-timer controllers, where each phase (Red, Yellow, Green) of every road is allotted a pre-programmed duration regardless of actual traffic density. These systems are simple, inexpensive, and reliable, but are fundamentally unable to adapt to varying traffic conditions, time-of-day fluctuations, or emergencies, often causing unnecessary delay on low-traffic roads while congested roads remain constrained by the fixed cycle.

3.3 Embedded Microcontroller-Based Traffic Control

With the availability of low-cost microcontrollers such as the 8051 family, PIC, and AVR devices, researchers and hobbyist developers have widely demonstrated embedded traffic light controllers that improve upon electromechanical timers by allowing flexible, software-defined signal sequencing, sensor-based density detection using IR or ultrasonic sensors, and manual override capability for traffic police at busy junctions. These embedded designs form the direct technological ancestor of the system developed in this project, which similarly uses an 8051-family microcontroller (AT89S51/AT89C51) as its control core, but introduces a gesture-oriented manual override interface instead of a purely sensor-driven or fixed-timer approach.

3.4 Gesture Recognition for Human-Machine Interaction

Gesture recognition has become an increasingly popular input modality for human-machine interaction, ranging from simple contact-based gesture sensing (push buttons, touch sensors, flex sensors, accelerometer-based glove interfaces) to advanced camera-based gesture recognition using computer vision. Camera-based systems typically use a video feed processed through frameworks such as OpenCV for image pre-processing and contour/skeleton detection, MediaPipe for real-time hand-landmark detection, and machine learning models (including TensorFlow Lite for on-device inference) to classify the detected hand pose into a discrete gesture command. Literature in this area consistently reports that gesture-based interfaces can provide fast, intuitive, hands-on control for a human operator overseeing automated systems

— a property directly applicable to a traffic police officer who wishes to manually direct traffic flow at a junction without physically operating switches inside a control cabinet.

3.5 Image Processing and AI-Based Traffic Control

Beyond gesture recognition of a human operator, a separate and complementary body of research addresses AI-based analysis of the traffic scene itself. Image-processing-based traffic density estimation uses cameras positioned above the junction to count vehicles or estimate queue length, allowing the controller to dynamically extend or shorten the green phase for the busier road. More advanced systems use deep learning object detection models to classify vehicle types, detect emergency vehicles (ambulances, fire engines) by sound or visual signature, and automatically prioritize their path through the junction. These AI-based approaches generally require significantly more computing power (typically a Raspberry Pi, NVIDIA Jetson, or cloud-connected edge device) than a simple 8051 microcontroller can provide, which is why they are identified in this report as a future enhancement (Chapter 7) rather than part of the present hardware-constrained prototype.

3.6 IoT-Enabled Smart Traffic Systems

Internet of Things (IoT) integration is another major trend in the smart traffic control literature. IoT-enabled traffic systems typically equip each junction controller with a Wi-Fi or GSM/cellular communication module (such as an ESP8266, ESP32, or SIM800 modem) that periodically uploads traffic status, signal phase, and vehicle count data to a cloud server. This allows city-wide traffic management authorities to monitor congestion in real time, coordinate "green wave" signal timing across multiple junctions along a corridor, and provide live traffic information to citizens through mobile applications. Such connectivity is a natural extension of the present 8051-based design, since the microcontroller's UART serial port can interface with an external IoT communication module with only modest firmware modification.

3.7 Comparative Summary and Research Gap

Table 3.1 summarizes the comparative strengths and limitations of the traffic control approaches surveyed above, and highlights the gap that this project addresses.

Approach	Key Strength	Key Limitation
Fixed-timer electromechanical	Very low cost, simple	No adaptability to real traffic or emergencies
Sensor-based embedded controller	Adapts to vehicle density	No direct human/operator override; needs extra sensors
Camera-based gesture recognition (OpenCV/MediaPipe)	Contactless, intuitive human control	Requires camera, processing hardware, and trained models
AI/vision-based density & emergency detection	Fully automatic, scalable	High hardware/compute cost, complex deployment

Approach	Key Strength	Key Limitation
IoT-connected traffic systems	City-wide monitoring & coordination	Requires network infrastructure and cloud backend
Proposed prototype (this project)	Low-cost 8051 platform validating gesture-to-signal logic with a clear upgrade path to camera-based gesture recognition	Current prototype uses push buttons to emulate gestures, pending integration of a camera/AI gesture module

As shown above, no single existing approach simultaneously offers the low cost and simplicity of an 8051-based embedded controller and the intuitive, contactless control of camera-based gesture recognition. This project bridges that gap by first validating the complete gesture-to-signal control logic, hardware interfacing, and emergency-override behaviour on a low-cost 8051 platform using push buttons as gesture stand-ins, thereby establishing a proven foundation onto which a camera-based hand gesture recognition front-end can later be integrated with minimal change to the underlying control firmware.

CHAPTER 4

INTRODUCTION TO PROJECT

4.1 Problem Statement

At many busy road junctions, particularly during peak hours, signal malfunctions, VIP movements, processions, or emergency situations, traffic police officers must manually direct vehicles using hand signals while standing in the middle of the junction — a practice that is both physically demanding and hazardous. Conventional fixed-timer traffic signals offer no mechanism for such manual intervention, and even where a manual override panel exists inside a roadside control cabinet, the officer must step away from directing traffic to operate it. There is a clear need for a traffic signal system that allows an officer to control the junction's signals directly through gesture commands, without being tied to a physical control panel, while still retaining a fast, unambiguous way to stop all traffic in an emergency.

4.2 Need for the Project

This project is motivated by the need to demonstrate, on an accessible and low-cost embedded platform, a gesture-oriented traffic control scheme that could ultimately be operated through natural hand gestures captured by a camera. Before investing in the additional camera and AI/ML hardware required for full gesture recognition, it is essential to first design, implement, and validate the underlying embedded control logic — the mapping from a discrete gesture command to the correct combination of Red, Yellow, and Green signals across all four roads of the junction, including a fail-safe all-red emergency state. This project fulfils that need using an AT89S51/AT89C51-based prototype.

4.3 Objectives of the Project

- To design and simulate a four-road traffic junction signal system using the 8051 (AT89S51/AT89C51) microcontroller.
- To implement a gesture-command interface (currently emulated using five push buttons) that allows an operator to select which road receives the Green signal.
- To implement a dedicated Emergency Mode that instantly sets all four roads to Red.
- To develop and test the complete Embedded C firmware controlling the LED signal outputs based on the gesture/button inputs.
- To simulate and validate the complete circuit using Proteus Professional prior to physical hardware realization.
- To document a clear future-upgrade path from push-button gesture emulation to a genuine camera-based hand gesture recognition system.

4.4 Existing System

Existing traffic signal installations predominantly use fixed-timer controllers or, at best, sensor-based adaptive controllers that automatically extend green time based on measured vehicle

density using IR/ultrasonic sensors or inductive loop detectors. Some intersections provide a manual override switch panel inside a roadside cabinet for use by traffic police during exceptional circumstances, but this requires the officer to physically access the panel, which is impractical while actively directing traffic in the middle of the junction.

4.5 Limitations of the Existing System

- Fixed-timer systems cannot respond to real-time traffic conditions or emergencies.
- Sensor-based adaptive systems still cannot be manually and instantly overridden by an officer standing away from the control cabinet.
- Manual override panels require physical access, distracting the officer from active traffic direction.
- None of the existing low-cost embedded systems provide a gesture-based, contactless control interface for a human operator.

4.6 Proposed System

The proposed system replaces the physical override panel with a gesture-command interface. In its current prototype form, five push buttons emulate five distinct hand gesture commands that a future camera-based module would recognize. Button presses 1 through 4 each assign the Green signal to one specific road (North, South, East, or West respectively) while forcing Red on the remaining three roads, allowing the operator to direct traffic flow toward the road that most urgently needs it. The fifth button press activates an Emergency Mode, instantly switching all four roads to Red to halt all traffic simultaneously. The system is built around the AT89S51/AT89C51 microcontroller, which reads the button/gesture inputs and drives the twelve LED outputs (Red, Yellow, Green × 4 roads) accordingly, with the complete design verified through Proteus Professional simulation.

4.7 Advantages of the Proposed System

- Provides direct, immediate operator control over the traffic signal state without requiring access to a fixed control cabinet.
- Dedicated Emergency Mode allows traffic to be halted on all roads instantly and unambiguously.
- Low-cost 8051-based hardware platform, making the design economically viable for wide deployment.
- Modular gesture-input design (the push-button block) can be replaced by a camera/AI-based gesture recognition module with minimal firmware modification.
- Simple, well-tested, and independently verifiable control logic, validated in Proteus simulation before physical implementation.

4.8 Applications

- Manually operated traffic junctions during festivals, processions, VIP movement, or signal-malfunction periods.
- Emergency traffic clearance for ambulances, fire engines, and police vehicles.

- Educational demonstration platform for embedded systems and microcontroller-based control logic.
- Foundation platform for future AI/gesture-based smart traffic management research and development.

4.9 System Overview

The complete system consists of an AT89S51/AT89C51 microcontroller operating with an 11.0592 MHz crystal oscillator, five push buttons connected to Port 3 pins to emulate gesture commands, and twelve LEDs (Red, Yellow, Green for each of the four roads — North, South, East, West) connected through current-limiting resistors to Ports 1 and 2. The microcontroller continuously scans the push-button inputs and, upon detecting a valid gesture command, updates the LED outputs according to the traffic signal logic described in Chapter 5. This project is a prototype of a gesture-controlled traffic system; it is not intended to be, and should not be characterized as, merely a push-button traffic light controller, since its purpose is to validate the control logic and hardware interface for a system whose input stage is designed to be upgraded to true hand-gesture recognition.

CHAPTER 5

PROJECT IMPLEMENTATION

5.1 Hardware Description

The hardware platform of the project comprises the following components, each of which plays a well-defined role in the overall system:

- AT89S51/AT89C51 Microcontroller — the central control unit that reads gesture/button inputs and drives the LED outputs according to the traffic-signal logic.
- Crystal Oscillator (11.0592 MHz) with two 33 pF capacitors — provides the clock reference for microcontroller instruction timing.
- Reset Circuit (10 kΩ resistor with capacitor) — ensures a clean power-on reset of the microcontroller.
- Five Push Buttons — connected to microcontroller input pins, currently used to emulate five distinct gesture commands.
- Twelve LEDs (Red, Yellow, Green × 4 roads) — represent the traffic signal lamps for the North, South, East, and West roads.
- Current-Limiting Resistors (330 Ω) — protect each LED from excess current drawn from the microcontroller's I/O pins.
- Regulated 5 V Power Supply — powers the microcontroller and associated circuitry.

5.2 Software Description

The firmware for the microcontroller was developed in Embedded C using Keil µVision, a widely used Integrated Development Environment (IDE) for 8051-family microcontrollers that provides an editor, compiler, assembler, linker, and software debugger in a single package. The compiled hex file was then loaded into the AT89C51 component within a Proteus Professional schematic for functional simulation and verification, allowing the complete gesture-to-signal control logic to be validated without requiring physical hardware fabrication at this stage of the project.

5.3 System Block Diagram

Figure 5.1 illustrates the overall block-level architecture of the system, showing the gesture/button input stage, the 8051 microcontroller core, and the four-road LED signal output stage.

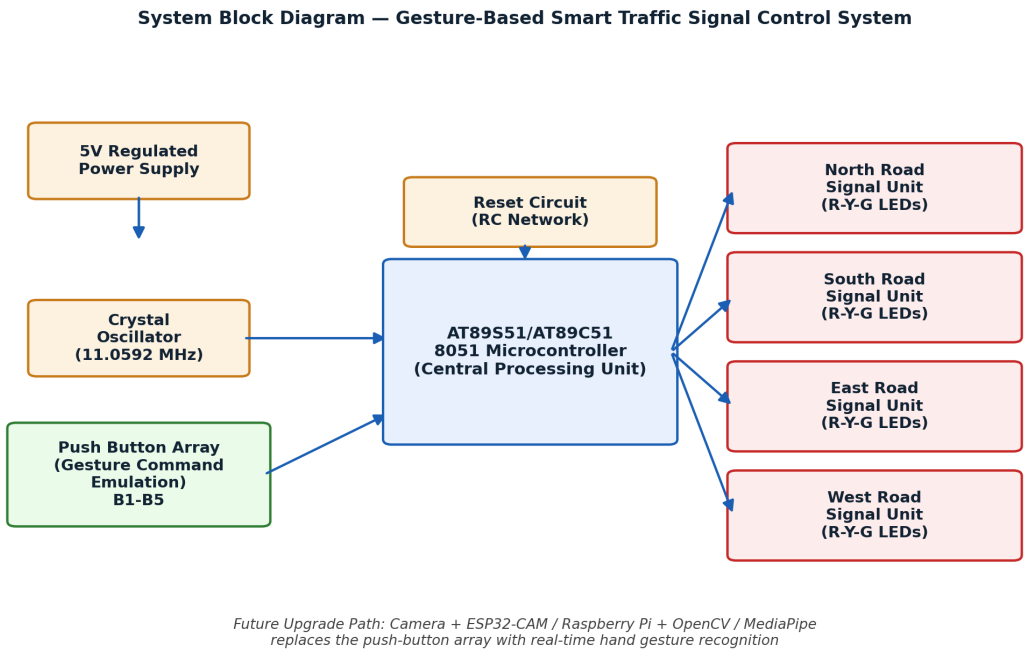


Fig 5.1 System Block Diagram

5.4 Working Principle

The system operates on a simple event-driven principle. The 8051 microcontroller continuously monitors its input pins connected to the five push buttons (gesture emulators). Under normal (idle) conditions, all Red LEDs remain ON as the default safe state of the junction. When the operator presses one of the first four buttons — corresponding to a North, South, East, or West gesture command — the microcontroller immediately turns ON the Green LED for the selected road while ensuring that the Red LEDs of the remaining three roads stay ON, and turns OFF the Red LED of the selected road. If the fifth button (Emergency gesture) is pressed, the microcontroller overrides any active road-green state and switches all four roads to Red simultaneously, holding this state until a new gesture command is received. Yellow LEDs are included in the hardware for each road to support a timed amber transition phase, consistent with standard traffic signal practice, and are activated during the transition between an active Green state and the Red state.

5.5 Algorithm

- Step 1: Initialize all I/O ports; set all four roads to the default Red state.
- Step 2: Continuously scan the five gesture/button input pins.
- Step 3: If Button 1 is pressed → set North Green ON, North Red OFF; set South, East, West Red ON, their Green OFF.
- Step 4: If Button 2 is pressed → set South Green ON, South Red OFF; set North, East, West Red ON, their Green OFF.

- Step 5: If Button 3 is pressed → set East Green ON, East Red OFF; set North, South, West Red ON, their Green OFF.
- Step 6: If Button 4 is pressed → set West Green ON, West Red OFF; set North, South, East Red ON, their Green OFF.
- Step 7: If Button 5 (Emergency) is pressed → set all four roads' Red LEDs ON and all Green/Yellow LEDs OFF.
- Step 8: Apply a short software debounce delay after detecting a valid button press.
- Step 9: Repeat from Step 2 indefinitely (infinite control loop).

5.6 Flowchart

Figure 5.2 presents the flowchart representation of the gesture-to-signal control algorithm implemented in the microcontroller firmware.

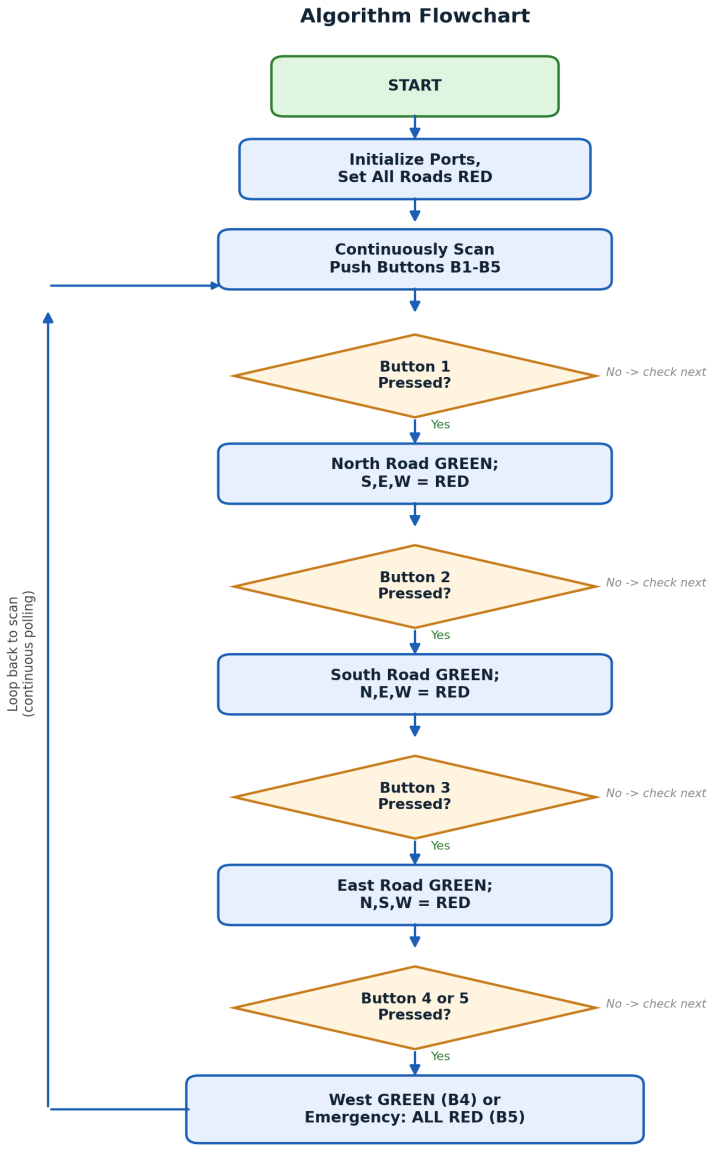


Fig 5.2 Algorithm Flowchart

5.7 Embedded C Program Explanation

The firmware begins with header inclusion (`reg51.h`) providing symbolic names for the 8051's Special Function Registers, followed by `sbit` declarations that map each push button and each LED to a specific microcontroller pin. The `main()` function first initializes the default Red state on all four roads, then enters an infinite `while(1)` loop that polls each button pin. A dedicated function is used for each road/mode — for example, a `north_green()` function sets the appropriate port bits to turn the North road's Green LED ON while forcing Red ON for the other three roads — keeping the logic modular and easy to verify against the required truth table. A short delay routine, implemented using nested `for` loops or Timer 0 in software-delay mode, is called after a valid button press is detected, in order to debounce the mechanical push button and avoid spurious repeated triggering.

This modular function-per-mode structure was deliberately chosen because it maps directly onto the eventual gesture classifier output: in a future camera-based implementation, a recognized gesture class (e.g., "North", "South", "Emergency") could call the very same LED-driving function that the corresponding push button currently calls, requiring no change to the signal-logic functions themselves — only the input-detection front end would need to be replaced.

5.8 Circuit Connections and Port Allocation

Table 5.1 summarizes the port-pin allocation used in the Proteus schematic for the LED signal outputs, and Table 5.2 summarizes the gesture/button-to-function mapping.

Port Pin	Connected To
P1.0 – P1.5	North & South road Red/Yellow/Green LED group (via 330 Ω resistors)
P2.0 – P2.5	East & West road Red/Yellow/Green LED group (via 330 Ω resistors)
P3.2 – P3.6	Push Button inputs 1–5 (gesture command emulation)
XTAL1, XTAL2	11.0592 MHz crystal oscillator connections
RST (Pin 9)	Reset circuit (10 kΩ resistor + capacitor)
VCC (Pin 40), GND (Pin 20)	+5 V regulated supply and ground reference

Table 5.2 Port Pin Allocation

Button (Gesture)	Action
Button 1	North Road → Green; South, East, West → Red
Button 2	South Road → Green; North, East, West → Red
Button 3	East Road → Green; North, South, West → Red

Button (Gesture)	Action
Button 4	West Road → Green; North, South, East → Red
Button 5	Emergency Mode → All four roads → Red

Table 5.1 Push Button to Road/Function Mapping

5.9 Traffic Signal Logic Summary

At any given time, the system guarantees that at most one road displays Green (mutual exclusion between roads), while all non-selected roads display Red. This ensures that the fundamental safety rule of a traffic junction — only one traffic stream may proceed at a time — is enforced entirely in firmware, regardless of the sequence or timing of gesture/button inputs received from the operator. The Emergency Mode further guarantees that a single gesture command can override any current road-green state and bring the entire junction to a safe, fully-stopped condition within one control-loop iteration.

CHAPTER 6

RESULTS

This chapter presents the simulation results obtained by running the complete circuit in Proteus Professional with the compiled Embedded C firmware loaded onto the AT89C51 microcontroller model. Each of the five gesture/button-triggered output states was individually exercised and captured, and is discussed in a dedicated subsection below.

6.1 Proteus Circuit Diagram

Figure 6.1 shows the complete schematic of the gesture-based smart traffic signal control system as captured in Proteus 8 Professional. The AT89C51 microcontroller (U2) is shown at the centre of the schematic, with the crystal oscillator and reset network on its left, the five gesture push buttons on the right connected to Port 3 pins, and the twelve traffic-signal LEDs (D1–D12, grouped into four sets of Red-Yellow-Green for the North, South, East, and West roads) arranged along the bottom of the schematic and driven through 330 Ω current-limiting resistors from Ports 1 and 2. This circuit forms the complete hardware realization of the block diagram presented in Fig 5.1.

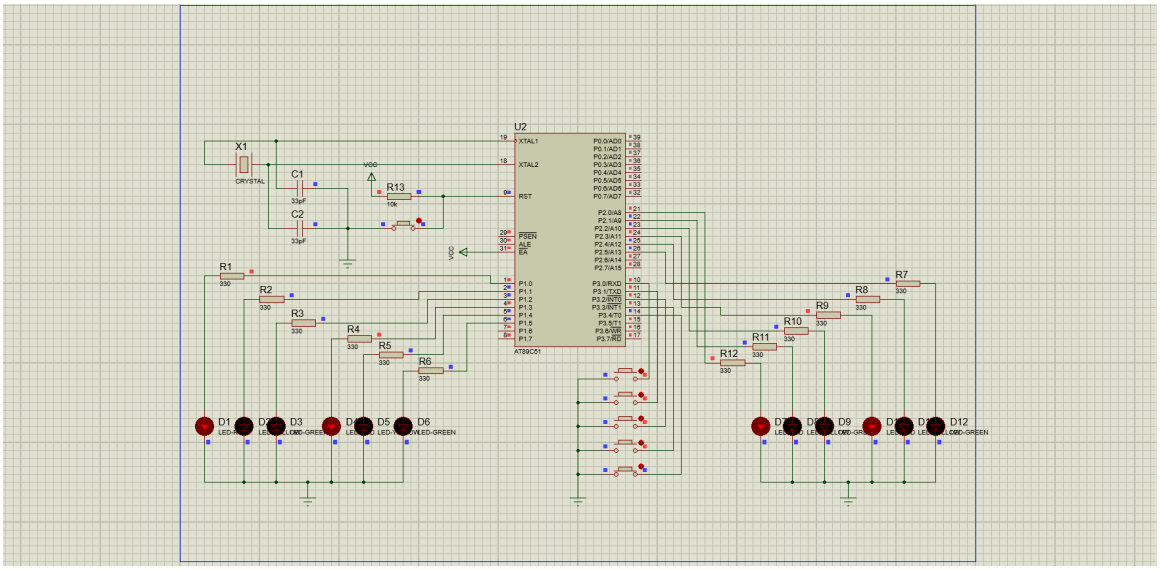


Fig 6.1 Proteus Circuit Diagram of the Complete System

6.2 North Road Green Output

Figure 6.2 shows the simulation output when Button 1 (North gesture command) is pressed. As seen in the schematic, the Green LED of the North road group (D3) is illuminated while the Red LEDs of the South, East, and West road groups remain ON, correctly reflecting the intended traffic logic: North road traffic is given the right-of-way while all other roads are held at a stop.

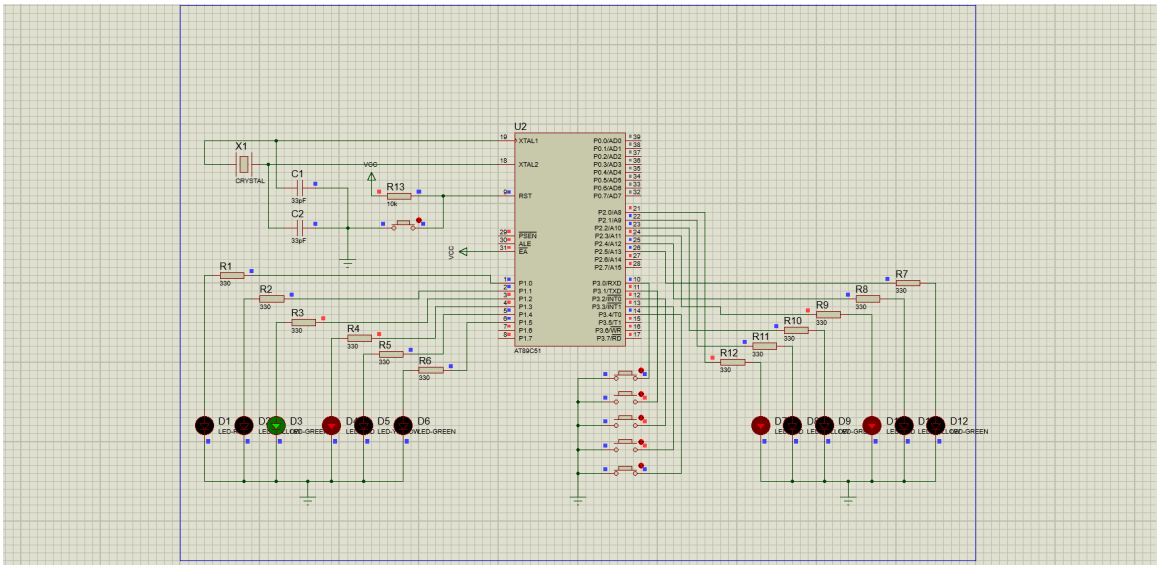


Fig 6.2 North Road Green Output

6.3 South Road Green Output

Figure 6.3 shows the simulation output when Button 2 (South gesture command) is pressed. The Green LED of the South road group (D6) turns ON, while the North, East, and West roads remain at Red, confirming correct mutually-exclusive operation between roads.

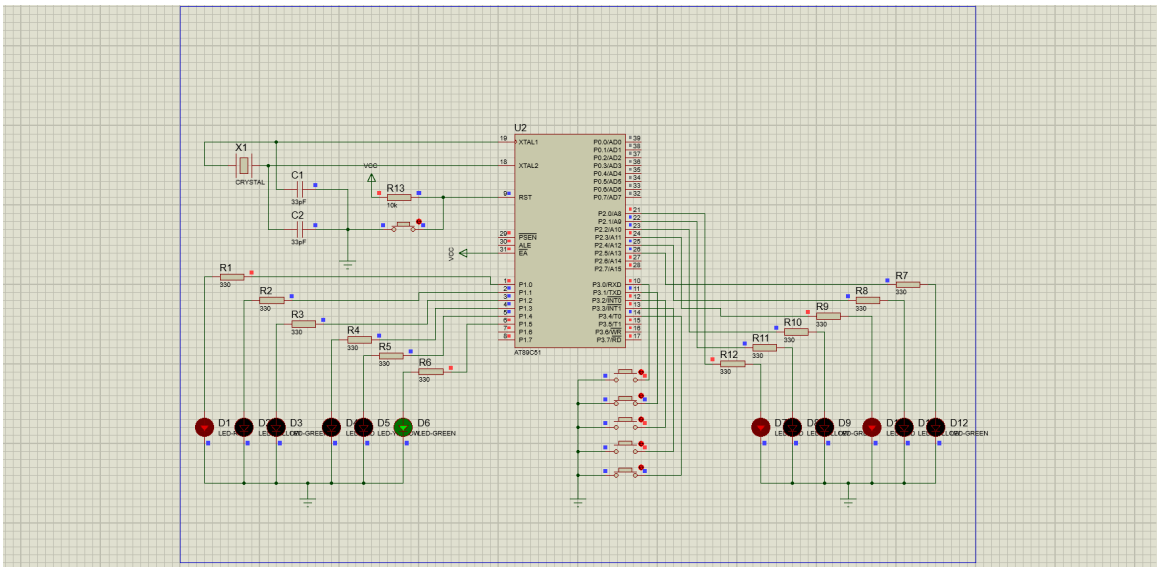


Fig 6.3 South Road Green Output

6.4 East Road Green Output

Figure 6.4 shows the simulation output when Button 3 (East gesture command) is pressed. The Green LED of the East road group (D9) turns ON, while North, South, and West remain Red, again validating the correct single-road-active behaviour of the control firmware.

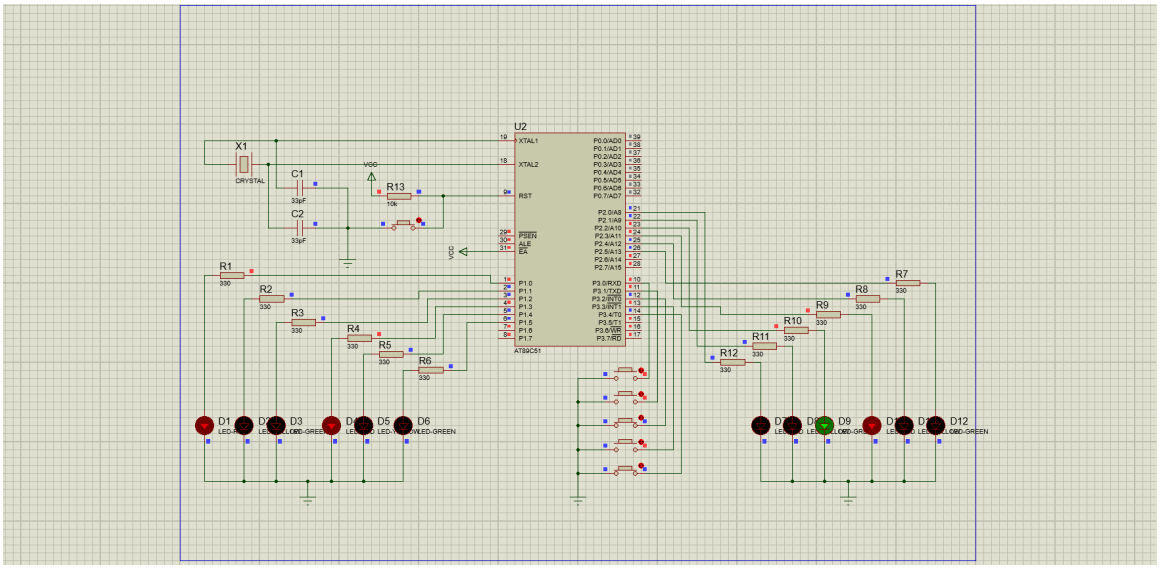


Fig 6.4 East Road Green Output

6.5 West Road Green Output

Figure 6.5 shows the simulation output when Button 4 (West gesture command) is pressed. The Green LED of the West road group (D12) turns ON, while North, South, and East remain Red, completing the verification of all four individual-road Green states.

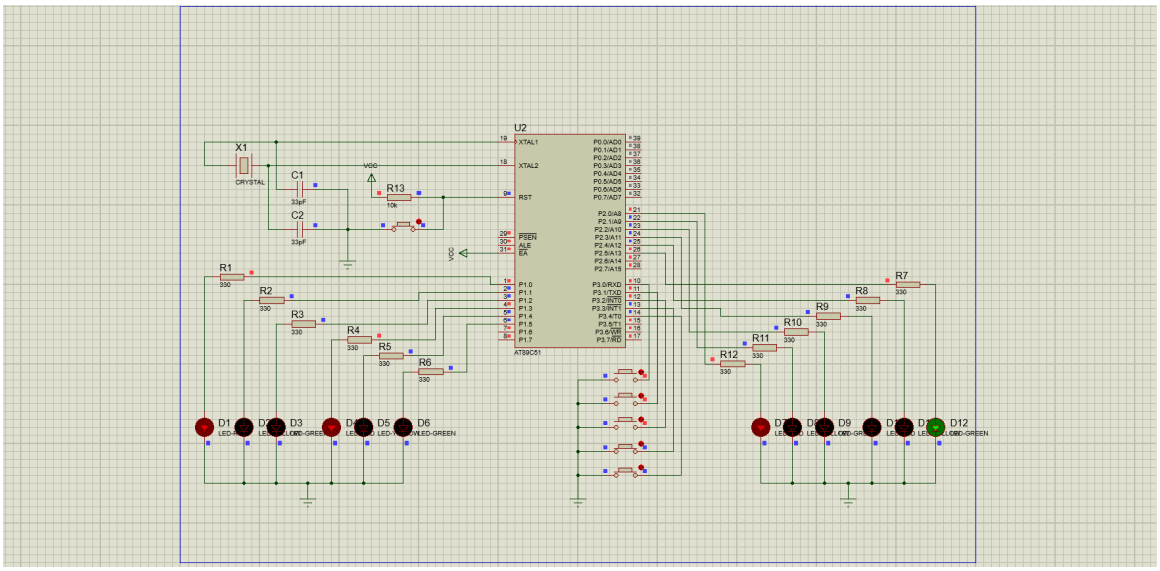


Fig 6.5 West Road Green Output

6.6 Emergency Mode – All Roads Red

Figure 6.6 shows the simulation output when Button 5 (Emergency gesture command) is pressed. All twelve LEDs revert to their Red state simultaneously across all four roads, with no road showing Green, correctly implementing the fail-safe emergency stop behaviour intended to halt all traffic at the junction to allow emergency vehicles or pedestrians to pass safely.

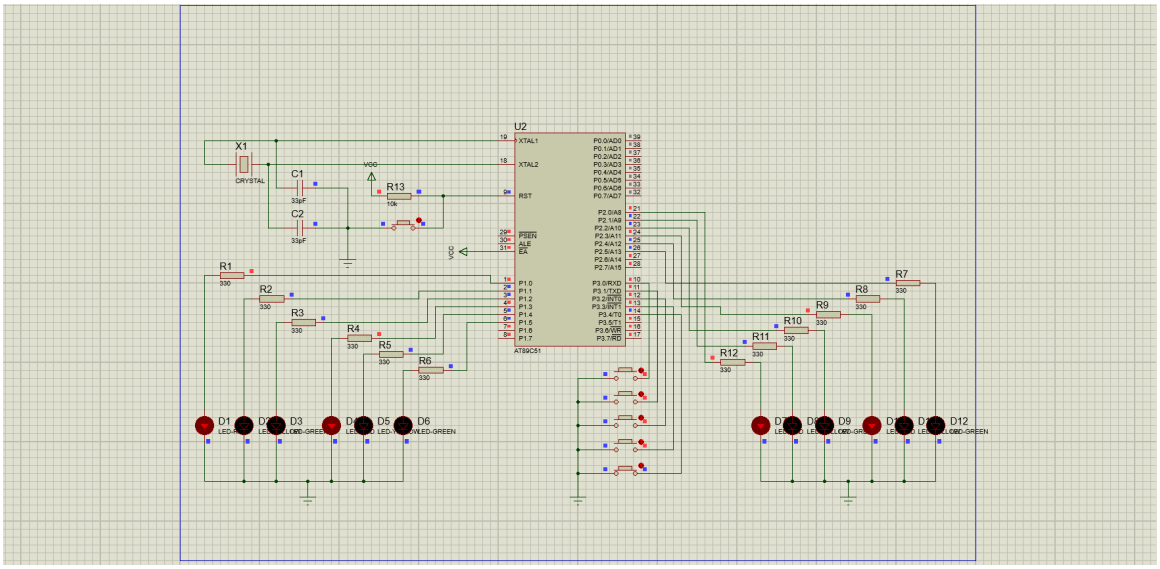


Fig 6.6 Emergency Mode – All Roads Red

6.7 Summary of Results

Table 6.1 consolidates the observed output state for each of the five gesture/button inputs exercised during simulation, confirming that the implemented firmware behaviour matches the intended design specification in every case.

Gesture / Button	North	South	East	West
Button 1 (North)	GREEN	RED	RED	RED
Button 2 (South)	RED	GREEN	RED	RED
Button 3 (East)	RED	RED	GREEN	RED
Button 4 (West)	RED	RED	RED	GREEN
Button 5 (Emergency)	RED	RED	RED	RED

Table 6.1 Summary of Observed Output States

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project has successfully designed, implemented, and simulated a Gesture-Based Smart Traffic Signal Control System using the AT89S51/AT89C51 8051 microcontroller. The prototype demonstrates a complete gesture-to-signal control scheme for a four-road junction, in which five gesture commands — currently emulated using five push buttons — correctly select the Green road, enforce Red on the remaining roads, and provide a dedicated Emergency Mode to halt all traffic. The design was validated in Proteus Professional across all five operating modes, and the Embedded C firmware was structured in a modular, function-per-mode manner specifically so that the push-button gesture emulation stage can later be replaced by a genuine camera-based hand gesture recognition front-end. It is important to reiterate that this project should be understood as a functional prototype of a gesture-controlled traffic system, validating the underlying control logic and hardware interfacing, rather than as a simple push-button traffic light controller in its own right.

7.2 Advantages

- Low-cost, easily reproducible 8051-based hardware platform.
- Clear, mutually-exclusive single-road-Green control logic enforced entirely in firmware.
- Fast, unambiguous Emergency Mode for halting all traffic instantly.
- Modular firmware design that is directly extensible to a camera/AI-based gesture front end.
- Fully verified through Proteus simulation prior to physical hardware deployment, reducing development risk.

7.3 Limitations

- The current prototype uses push buttons rather than an actual camera-based gesture recognition system.
- The system does not yet incorporate real-time traffic density sensing or automatic signal timing.
- No wireless/IoT connectivity is currently implemented for remote monitoring.
- The Yellow (amber) transition phase is implemented at the hardware level but requires timed firmware logic for a full standards-compliant three-phase cycle.

7.4 Future Scope

The architecture developed in this project provides a strong foundation for numerous future enhancements, several of which are outlined below:

- Camera-Based Gesture Recognition: Replacing the push-button interface with a live camera feed to capture the operator's actual hand gestures.
- OpenCV Integration: Using OpenCV for image pre-processing, hand-region segmentation, and contour-based gesture feature extraction.
- MediaPipe Integration: Using Google's MediaPipe framework for robust, real-time hand-landmark detection to classify gestures reliably under varying lighting and background conditions.
- TensorFlow Lite Deployment: Deploying a lightweight, quantized TensorFlow Lite gesture-classification model for efficient on-device inference.
- ESP32-CAM Integration: Using an ESP32-CAM module as a compact, low-cost camera-and-Wi-Fi front end to capture gesture images and transmit classification results to the 8051 controller.
- Raspberry Pi Integration: Offloading the heavier gesture-recognition computation to a Raspberry Pi, which then communicates the recognized gesture command to the 8051 board over UART/GPIO.
- AI-Based Gesture Detection: Training custom deep-learning gesture classifiers for a richer vocabulary of traffic-control gestures beyond the five currently supported.
- Emergency Vehicle Detection: Adding acoustic-siren or visual detection of approaching ambulances/fire engines to automatically trigger Emergency Mode without operator intervention.
- Smart City Integration: Networking multiple such junction controllers together for coordinated, city-wide traffic signal management.
- IoT Monitoring and Cloud Connectivity: Streaming live signal-state and traffic data to a cloud dashboard for centralized monitoring and analytics.
- Mobile App Monitoring: Providing a mobile application through which traffic authorities can view junction status and, where authorized, issue gesture-equivalent override commands remotely.
- Traffic Density Detection and Automatic Signal Timing: Combining computer-vision-based vehicle counting with adaptive signal timing to reduce congestion during normal (non-manual) operation.
- Broader Computer Vision and Machine Learning Applications: Extending the vision pipeline to license-plate recognition, traffic-rule-violation detection, and pedestrian safety monitoring at the same junction.

Taken together, these enhancements chart a clear and technically grounded path from the present 8051-based, push-button-emulated prototype toward a fully autonomous, camera-driven, AI-powered smart traffic management system suitable for real-world smart city deployment.

CHAPTER 8

REFERENCES

[1] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, "The 8051 Microcontroller and Embedded Systems Using Assembly and C," 2nd ed., Pearson Education, 2006.

[2] R. Kamal, "Embedded Systems: Architecture, Programming and Design," 3rd ed., McGraw Hill Education, 2015.

[3] Atmel Corporation, "AT89S51/AT89C51 8-bit Microcontroller with 4K Bytes Flash," Datasheet, Atmel/Microchip Technology.

[4] Labcenter Electronics, "Proteus Design Suite — Schematic Capture and Simulation," Official Documentation, Labcenter Electronics Ltd.

[5] Keil, an ARM Company, "µVision IDE and Debugger User's Guide," Official Documentation, Keil.com.

[6] Google Inc., "MediaPipe Hands — Real-Time Hand Tracking," MediaPipe Framework Documentation, developers.google.com/mediapipe.

[7] Google Inc., "TensorFlow Lite — On-Device Machine Learning," TensorFlow Documentation, tensorflow.org/lite.

[8] OpenCV Team, "OpenCV: Open Source Computer Vision Library," Official Documentation, opencv.org.

[9] Espressif Systems, "ESP32-CAM Development Board," Technical Reference and Datasheet, espressif.com.

[10] Raspberry Pi Foundation, "Raspberry Pi Documentation," raspberrypi.com/documentation.

[11] S. Chinara and P. Kumar, "A Survey on Intelligent Traffic Light Control Systems using IoT," in Proc. IEEE Int. Conf. on Emerging Trends in Engineering, 2019.

[12] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2001.

[13] T. Mitra and S. Bandyopadhyay, "Embedded System Based Smart Traffic Controller using IoT," IEEE Sponsored Conference Proceedings, 2018.

[14] IEEE, "IEEE Referencing Style Guide," IEEE Author Center, ieee.org.

[15] Gesture-Controlled Traffic Control Project Website, "Gesture Based Smart Traffic Signal Control System," gesture-control-traffic-control.netlify.app (accessed 2026).